

Translator API Usage Guide

Translation API — Quick Usage Guide

Companion reference for the translator at `nginx:3101` (OpenAI-compat) and `:3102` (translator / `translate` endpoint). Pick the minimum set of params for your call context.

Minimum required

```
POST /translate
{ "text": "...", "source_lang": "en", "target_lang": "zhTW" }
```

Everything else is optional. Pipeline auto-detects `content_type`, `intent`, `teaching_lang`, `loanword_tolerance`, `vocab_mode` from text signals. Good for "just translate this" use cases.

For pedagogical / language-learning content (subtitles, lesson material)

Always pass these explicitly — auto-detection isn't perfect:

```
{
  "text": "...",
  "source_lang": "en", "target_lang": "zhTW",
  "teaching_lang": "English",           // tells pipeline English tokens must
survive
  "content_type": "subtitle",           // or "ui_button", "ui_heading"
  "domain": "education",               // unlocks education glossary +
pedagogical intent
  "intent": "pedagogical",             // consistent revise rules
  "context": "Subtitle from an English-language lesson video for language
learners."
}
```

- `teaching_lang=English` activates Pattern 2 preservation (English vocab stays in Latin script). Default `vocab_mode` = `"preserve"` (no target-lang gloss).
- If you need `word` (translation) bilingual output, pass `"vocab_mode": "bilingual"` explicitly.

- For source text containing `{{double braced}}` tokens (Pattern 1), no extra param needed — pipeline preserves content + strips braces from delivered output.

For legal / marketing / UI microcopy

Keep `content_type` explicit (`legal` | `cta` | `ui_button` | `ui_heading`). Pass `intent` (`legal` | `transcreation_marketing` | `transcreation_ux`) for strictness-appropriate revise behavior. Legal: `"strictness": "linguist"` for maximum fidelity checks.

Speaker-aware subtitle

When dialogue gender matters (Thai/Japanese/Korean particles):

```
"speaker": { "gender": "female", "role": "teacher", "age_band": "adult" }
```

Preserve-verbatim brand names / technical terms

```
"preserve_list": ["FlowTasks", "TensorFlow", "kubect1"]
```

These never translate, regardless of context. Pipeline wraps them in `{{...}}` internally and strips braces at delivery.

What's auto-detected vs what to pass

Param	Auto-detected?	When to override
<code>content_type</code>	Yes (from length, punctuation, timestamp patterns)	Always for pedagogical (auto picks <code>subtitle</code> reliably but fails on mixed).
<code>intent</code>	Yes	Override for marketing/legal — auto defaults to <code>literal</code> .
<code>teaching_lang</code>	Yes (from text signals)	Override when text is ambiguous (short sentences).

<code>vocab_mode</code>	Defaults to preserve when <code>teaching_lang</code> set	Pass <code>bilingual</code> for side-by-side output.
<code>loanword_tolerance</code>	Per-lang default	Override for strict nativization (<code>low</code>) or permissive (<code>high</code>).
<code>domain</code>	Not auto-detected	Pass "education", "legal", "tech", "medical", "finance", "marketing" for glossary routing.

Response fields to read

- `translation` — delivered text (braces stripped, S5-selected)
- `confidence` — `high` / `medium` / `low`
- `audit.stage_timings.s5_decision` — which candidate was delivered
- `audit.stage_timings.s5_override_final` — true when S5 reverted an S4 decision
- `audit.stage_timings.had_revision` — true when S4 fired
- `warnings` — non-empty only when something degraded (FALLBACK_ORIGINAL, placeholder leak, etc.)

Cache semantics

- Default: reads cache first, writes on miss.
- `no_cache: true`: skips both read AND write (true fresh pipeline run).
- Cache key = `hash(source_lang, target_lang, source_text, context, teaching_lang, vocab_mode)`. Different `context` strings = different cache entries.

Endpoints

- `POST /translate` — single translation
- `POST /translate/batch` — batch translate (auto cross-segment context for subtitles)
- `POST /review` — review a human translation vs source
- `POST /review/batch` — batch review
- `POST /tm/add` / `/tm/bulk` — translation memory ingestion

3 preservation patterns (contract)

- Pattern 1: `{{double-braced}}` source tokens → content survives, braces stripped at delivery.
- Pattern 2: unbraced English teaching tokens (when `teaching_lang` set) → stay in Latin script, no translation.
- Pattern 3: delivered text has no braces, content preserved.

See `bench-method.md` § Preservation Patterns for the authoritative spec used by both pipeline and Opus judge.

`#translator`

`#api`

`#reference`